BLACKOUT-CITY // SICHERHEITSARCHITEKTUR & TESTPHASE

Schlachtplan: Individuelle Beta-Codes & Discord-Bot

Verbindliche Spezifikation zur Absicherung der geschlossenen Testphase für **blackout-city.de**: Einmalige Einladungstoken, Discord-Bot-Generierung, API-Einlösung und UI-Schutz vor Informationslecks.

1. Ausgangslage & Problemstellung des bisherigen Setups

Im bisherigen Entwicklungsstand wurde der Schutz der Alpha-Version über ein einzelnes, statisches Passwort (VITE_BETA_ACCESS_PASSWORD) in einer Vorschaltmaske (BetaGate.jsx) realisiert. Obwohl dieser Mechanismus unautorisierte Zufallsbesucher abfängt, offenbart die Analyse für eine erweiterte Testgruppe entscheidende Schwachstellen:

Schwachstelle / Risiko	Bisheriger Zustand (Single Password)	Auswirkung auf den Testbetrieb & Sicherheit
Passwort-Weitergabe	Ein universeller Code für alle Tester.	Wird der Code einmal geteilt, verliert das Entwicklerteam die Kontrolle über den Nutzerkreis.
Resend-Kontingentschutz	Registrierung ohne Vorab-Prüfung möglich.	Bösartige oder ungeduldige Nutzer könnten E-Mail-Registrierungen spammen und das Monatskontingent verbrauchen.
Assets- & UI-Sicherheit	Gate sperrt nur die Startseite.	Eingeloggte Nutzer ohne echte Spielberechtigung könnten sensible Spielmechaniken oder Artwork-Assets vorab einsehen.
Manuelle Steuerung	Kein Verwaltungswerkzeug für Zugänge.	Zulassungen erfordern Code- oder .env-Updates und Server-Neustarts auf Render.

2. Die Zielarchitektur: Discord-Bot & Unique-Code-Pipeline

Das neue Sicherheitskonzept trennt die Account-Erstellung bewusst von der **Spielzugangsberechtigung (Dome Clearance)**. Registrierung und E-Mail-Bestätigung laufen wie geplant ab, das eigentliche Spielgeschehen wird jedoch erst nach Eingabe eines **einmaligen, kryptografisch generierten Beta-Codes** freigeschaltet:

Systemkomponente	Technologische Umsetzung	Zweck & Sicherheitsfunktion
Discord-Bot (Admin)	discord.js im Backend integriert	Generiert auf Slash-Befehl /betacode frische Einmal-Token direkt im privaten Admin-Kanal.
MongoDB (BetaCodes)	Neue Collection BetaCode.js	Verwaltet Token-Status (code, isUsed, createdFor, usedBy, expiresAt).
Player-Modell	Flag hasBetaAccess: Boolean	Definiert serverseitig und persistent, ob ein Account die Spiel-Loop betreten darf.
Redeem-API	POST /api/auth/redeem-beta	Validiert den Code atomar, markiert ihn als verbraucht und schaltet den Spieler dauerhaft frei.
Frontend-Isolierung	Restriktives BetaGate.jsx	Hält Spielkomponenten (ResourceBar, BuildingCard, Assets) vollständig ungerendert zurück.

Kernvorteil der Discord-Bot-Lösung: Das kostenlose monatliche E-Mail-Kontingent bei Resend (3.000 Mails) wird geschont, da Codes nicht blind per Mail versendet, sondern gezielt per Discord-Chat an handverlesene Tester verteilt werden.

3. Phasenübersicht des Implementierungsplans

Die Einführung des erweiterten Beta-Systems gliedert sich in fünf präzise Phasen, die sich unabhängig voneinander im bestehenden MERN-Stack testen lassen:

Phase	Bezeichnung & Fokus	Kernaufgaben & Technische Deliverables
Phase 1	Datenmodellierung & MongoDB	Erstellung des BetaCode-Schemas & Erweiterung des Player-Schemas um hasBetaAccess.
Phase 2	Discord-Bot-Integration	Einrichtung des Bots mit discord.js, Slash-Command /betacode & DB-Speicherung.
Phase 3	Backend-API (Redeem-Flow)	Controller redeemBetaCode, atomare Validierung & Schutz vor Race Conditions.
Phase 4	Frontend Gate & Absicherung	Umbau von BetaGate.jsx zur Code-Eingabe & restriktive Weiche in App.jsx.
Phase 5	Testing & Render-Deployment	Umgebungsvariablen für Discord-Token, lokaler Smoke-Test & Live-Gang auf blackout-city.de.

Phase 1: Datenmodellierung (Mongoose)

In server/src/models/BetaCode.js wird eine eigenständige Collection für Zugangscodes eingerichtet. Gleichzeitig wird server/src/models/Player.js erweitert:

Modell & Pfad	Felder & Datentypen	Validierung & Zweck
BetaCode.js (Neu)	code: String (unique) isUsed: Boolean (default: false) usedBy: ObjectId (ref: Player) usedAt: Date note: String (z.B. Tester-Name)	Speichert den Einmalcode. Index auf code sorgt für blitzschnelle Validierung und verhindert Duplikate.
Player.js (Erweiterung)	hasBetaAccess: Boolean (default: false) betaRedeemedAt: Date	Zentraler Schalter für Spielberechtigung. Ohne hasBetaAccess === true hat der Spieler keinen Zugriff auf Spieldaten.

Phase 2: Discord-Bot zur Code-Generierung

Der Discord-Bot läuft als leichtgewichtiger Service direkt im Node.js-Backend mit und greift auf dieselbe MongoDB-Instanz zu:

- **Paket-Installation:** npm install discord.js im Ordner server.
- **Bot-Modul (server/src/services/discordBot.js):** Initialisierung mit Client({ intents: [GatewayIntentBits.Guilds] }).
- **Slash-Befehl /betacode [notiz]:** Nur autorisierte Discord-Rollen (Admin) dürfen den Befehl ausführen. Der Bot erzeugt einen gut lesbaren Sci-Fi-Code (z.B. BC-7X9K-2026 via crypto.randomBytes), speichert ihn in MongoDB mit isUsed: false und antwortet im Discord mit einer formatierten Embed-Nachricht.
- **Konfiguration:** DISCORD_BOT_TOKEN und DISCORD_CLIENT_ID in server/.env.

Phase 3: Backend-API & Einlöse-Logik

Die Einlösung des Beta-Codes erfolgt über einen geschützten API-Endpunkt, der an die bestehende JWT-Authentifizierung gekoppelt ist:

Route & Methode	Header & Payload	Controller-Logik & Sicherheitsprüfungen	Erfolgsantwort
POST /api/auth/redeem-beta	Header: Authorization: Bearer JWT Body: { code: "BC-7X9K-2026" }	1. Spieler per JWT identifizieren. 2. Prüfen, ob Spieler bereits hasBetaAccess === true hat. 3. BetaCode suchen mit { code, isUsed: false }. 4. Atomares Update: Code auf isUsed: true und usedBy: player._id setzen. 5. Spieler auf hasBetaAccess: true setzen.	**200 OK** { success: true, message: "Zugang autorisiert!", hasBetaAccess: true }

Sicherheits-Check (Anti-Race-Condition): Die Einlösung muss mit findOneAndUpdate({ code, isUsed: false }, { isUsed: true, usedBy: req.userId }) ausgeführt werden. So wird ausgeschlossen, dass zwei Anfragen denselben Code zeitgleich verbrauchen.

Phase 4: Frontend-Gate & Schutz vor Informationslecks

Ein zentrales Anliegen ist es, dass unberechtigte Nutzer weder Einblick in die Spielmechaniken erhalten noch sensible Grafiken oder Ressourcenwerte laden können. Die Komponenten-Weiche in client/src/App.jsx wird daher präzise gestaffelt:

Nutzer-Status	Was gerendert wird	Sicherheits-Status & Informationsschutz
1. Nicht eingeloggt	LandingPage.jsx & AuthModal.jsx	Öffentliche Vorstellung, Registrierung & Login. Keinerlei Spiel-Assets im Speicher.
2. Eingeloggt, E-Mail offen	VerifyEmailHint.jsx	Aufforderung zur Bestätigung der Bürger-ID via E-Mail. Kein Zugang zum Spiel.
3. Eingeloggt, kein Beta-Code	BetaGate.jsx (Code-Eingabemaske)	**Geschlossenes Tor:** Nur Eingabefeld für den Beta-Code. Spiel-Loop, ResourceBar und BuildingCard existieren nicht im DOM.
4. Berechtigter Beta-Tester	App.jsx (Vollständige Game-Loop)	Voller Zugriff: Ressourcen-Produktion, Gebäude-Upgrades und Sci-Fi-Artwork.

UI/UX-Gestaltung von BetaGate.jsx:

- **Atmosphärisches Kuppel-Terminal:** Dunkler Hintergrund (#020617), dezenter Cyan-Glow (#06b6d4) und Statustext: „Bürger-ID bestätigt. Kuppel-Sektor gesperrt – Autorisierungscode erforderlich.“
- **Formular:** Großes Eingabefeld mit Platzhalter BC-XXXX-2026, automatischer Großschreibung (text-transform: uppercase) und Einlöse-Button.
- **Feedback:** Grüne Bestätigung mit Sound-/Animationseffekt bei Erfolg; klare Fehlermeldung bei ungültigem oder verbrauchtem Code.
- **Abmelde-Möglichkeit:** Ein diskreter „Abmelden“-Button, falls der Tester einen anderen Account nutzen möchte.

Phase 5: Umgebungsvariablen & Deployment-Konfiguration

Für den reibungslosen Betrieb auf Render und die lokale Entwicklung werden die Konfigurationsparameter harmonisiert:

Variable / Key	Zielumgebung	Beschreibung & Wertbeispiel
MONGO_URI	Server (.env & Render)	Verbindung zur MongoDB Atlas Datenbank.
JWT_SECRET	Server (.env & Render)	Kryptografischer Schlüssel zur Signierung der Login-Sessions.
RESEND_API_KEY	Server (.env & Render)	Versand von Registrierungs- und Bestätigungs-Mails über HTTPS.
EMAIL_FROM	Server (.env & Render)	Blackout-City <noreply@blackout-city.de>
DISCORD_BOT_TOKEN	Server (.env & Render)	Bot-Authentifizierungstoken aus dem Discord Developer Portal.
DISCORD_CLIENT_ID	Server (.env & Render)	Applikations-ID des Discord-Bots zur Befehlsregistrierung.
DISCORD_ADMIN_CHANNEL	Server (.env & Render)	Kanal-ID des geschützten Discord-Kanals für Code-Generierung.

6. Zusammenfassung des Benutzer-Workflows

Der vollständige Pfad eines Testers gestaltet sich damit hochprofessionell, nachvollziehbar und manipulationssicher:

1. **Code-Generierung:** Du gibst im privaten Discord-Kanal /betacode tester_alex ein. Der Bot antwortet mit BC-K9P2-2026.
2. **Code-Übergabe:** Du sendest den Code via Discord-DM oder Nachricht an deinen Tester.
3. **Registrierung:** Der Tester besucht blackout-city.de, erstellt seinen Account und bestätigt seine E-Mail-Adresse via Resend-Link.
4. **Login & Gate:** Nach dem Login landet der Tester automatisch vor dem geschlossenen Terminal (BetaGate.jsx).
5. **Freischaltung:** Der Tester gibt BC-K9P2-2026 ein. Das Backend entwertet den Code, setzt hasBetaAccess: true und das Kuppel-Interface öffnet sich.
6. **Dauerhafter Zugang:** Bei allen künftigen Logins betritt der Spieler direkt das Spiel – der Code wird nie wieder abgefragt.

Checkliste vor dem Live-Gang:
[X] Discord-Application im Developer Portal erstellen & Bot-Berechtigungen (Slash Commands) vergeben.
[X] Mongoose-Modell BetaCode.js und Route /api/auth/redeem-beta bereitstellen.
[X] BetaGate.jsx in App.jsx als Zwischenschritt einbinden.
[X] DISCORD_BOT_TOKEN im Render Dashboard hinterlegen.